Waypoint

Employer Job Posting Implementation Guide

A simple explanation of how the frontend, backend, MongoDB, authentication, testing, and Git branch fit together.

What was delivered
An employer can register, verify an OTP, log in, receive a JWT, open the Job Postings page, generate a Job ID, publish a job, see the saved job in the list, edit active jobs, and close or reopen them.

Branch delivered: **Employer/JobPosting-NewPosting**

Commit: **1b17b52** | main was not changed.

1. How to run it

There are two development processes, but the browser uses one URL. Vite owns port 5173 and forwards API calls to Spring Boot on port 8081. Spring Boot connects to MongoDB Atlas using the MONGODB_URI environment variable.

Part	Open or run	Purpose
Frontend	http://localhost:5173	Employer screens and forms
Job list	http://localhost:5173/employer/jobs	View, search, filter, edit, close, and reopen jobs
New posting	http://localhost:5173/employer/jobs/new	Open the complete new-job form
Backend	http://localhost:8081	Receives API requests and applies business rules
Database	MongoDB Atlas, database waypoint	Stores users, profiles, OTP records, and jobs

To start the frontend, run **npm run dev** inside the frontend folder. Start BackendApplication from IntelliJ with the MongoDB connection environment variable. Do not start a second backend process while one is already using port 8081.

The root route sends a new visitor to employer registration. After employer login, the app sends the user directly to the Job Postings page.

Important idea: the browser does not call MongoDB directly. Only Spring Boot talks to MongoDB.

2. The frontend: what the employer sees

The main screen is implemented in **frontend/src/pages/JobPostingPage.jsx**. It is both the job list screen and the new/edit posting screen.

UI action	What the frontend does
Open Job Postings	Calls GET /api/employer/jobs and renders the employer's jobs.
Click + New posting	Opens the full form and requests a new Job ID from the backend.
Type in Search jobs	Filters the loaded list immediately by ID, title, department, location, work type, employment type, experience, or skill.
Choose Filter	Shows only Active or Closed postings when selected.
Click Edit	Loads an active posting into the form and sends a PUT request when saved.
Click Close/Reopen	Sends a status request and updates the row after the backend responds.

The shared Axios client uses a relative base URL of **/api**. That is why the page stays on localhost:5173 while Vite forwards API traffic to Spring Boot.

```
const apiClient = axios.create({ baseURL: "/api" });

// Every authenticated request receives the JWT automatically.
config.headers.Authorization = `Bearer ${token}`;
```

3. The backend: where the rules live

The backend separates responsibilities so the browser stays simple:

- JobController receives HTTP requests, reads the Bearer token, and returns a consistent ApiResponse.
- JobService applies ownership checks, validation, Job ID generation, status changes, and persistence.
- CreateJobRequest validates the incoming fields before the service runs.
- Job is the MongoDB document model and JobRepository reads and writes the jobs collection.
- EmployerJobResponse is the safe response shape returned to the React page.

Job API endpoints

Method	Browser path	Meaning
GET	/api/employer/jobs	List jobs owned by the logged-in employer
GET	/api/employer/jobs/new-code	Generate an available JOB-XXXXXXXX code
POST	/api/employer/jobs	Create and save a new posting
PUT	/api/employer/jobs/{jobId}	Update an active posting
PATCH	/api/employer/jobs/{jobId}/close	Close an active posting
PATCH	/api/employer/jobs/{jobId}/reopen	Reopen a closed posting

What happens when Publish posting is clicked

Step	Backend action
1	Extract the JWT and confirm the role is EMPLOYER.
2	Find the employer profile using the user ID from the token.
3	Validate required fields, salary order, skills, and the deadline.
4	Use the requested Job ID or generate a unique JOB-XXXXXXXX code.
5	Set the new status to OPEN and save the Job document.
6	Return the saved job, including its ID, code, status, and timestamps.

Date rule
The calendar minimum is the local current date. The frontend rejects a past date, and the backend also uses FutureOrPresent plus a service check. A request cannot bypass this rule by editing browser HTML.

4. MongoDB: what is stored

MongoDB stores documents. The backend maps Java entities to collections. The employer's user ID is linked to an employer profile, and the job stores the employer profile ID. This is how one employer sees only their own postings.

Collection	Important fields	Why it exists
users	_id, email, passwordHash, role, isEmailVerified, accountStatus	Login identity and account state
employer_profiles	_id, userId, companyName, address, phone, companySize	Employer details and ownership link
otp_verifications	email, otpHash, purpose, expiresAt, isVerified	One-time email verification state
jobs	_id, employerProfileId, jobCode, title, skills, salary, deadline, status	The actual job posting

Example job document

```
{
  "_id": "generated-id",
  "employerProfileId": "employer-profile-id",
  "jobCode": "JOB-A1B2C3D4",
  "title": "Java Software Engineer",
  "department": "Engineering",
  "workMode": "HYBRID",
  "requiredSkills": ["Java", "Spring Boot", "MongoDB"],
  "applicationDeadline": "2026-08-15T23:59:59",
  "status": "OPEN"
}
```

The frontend does not write directly to MongoDB. After a successful POST response, the page inserts the returned job into its visible list. Refreshing the page calls GET again, so the list comes from MongoDB rather than only browser memory.

Ownership protection
The service compares the token's employer user ID to the employer profile and then compares the job's employerProfileId. An employer cannot edit or close another employer's job by changing the URL.

5. Registration, OTP, and JWT

Authentication must happen before job APIs are allowed. The flow is deliberately separate: registration creates the account, OTP verification activates email verification, and login creates the JWT session.

Method	Path	Result
POST	/api/auth/register/employer	Creates user/profile and generates a six-digit OTP
POST	/api/auth/verify-otp	Checks the OTP and marks the email verified
POST	/api/auth/login	Returns accessToken, role, and account information

OTP behavior

- The OTP is six digits and valid for 10 minutes.

- MongoDB stores a BCrypt hash, not the raw OTP.

- For this local setup, the raw OTP is printed in the backend IntelliJ log by DevOnlyOtpLogger.

- The frontend no longer displays a development OTP banner or puts the code in the URL.

JWT behavior

- Login creates a signed Bearer token with user ID, email, role, issue time, and expiration.

- The current expiration is 24 hours.

- The Axios interceptor reads accessToken from localStorage and adds Authorization: Bearer ... to job requests.

```
Registration -> OTP log in IntelliJ -> verify OTP -> login
       -> JWT in browser -> authenticated job request
       -> controller checks role -> service writes MongoDB
```

Production reminder
Use a strong JWT_SECRET and keep EXPOSE_DEV_OTP=false outside local development. Never rely on the development log as a real email delivery system.

6. Testing and delivery

The implementation was checked at three levels:

Check	Command	What it proves
Backend compile	mvnw.cmd -q -DskipTests compile	Java code and configuration compile
Job service tests	mvnw.cmd -q -Dtest=JobServiceTest test	Job rules and service behavior pass targeted tests
Frontend lint	npm run lint	React/JavaScript style and syntax checks pass
Frontend build	npm run build	Vite can produce the deployable frontend bundle
Live round trip	register -> OTP -> verify -> login -> jobs	Requests, responses, JWT, proxy, and MongoDB work together

The live round trip was completed with a temporary employer account. The test account and its temporary profile/OTP data were removed afterward.

Git delivery

Item	Value
Branch	Employer/JobPosting-NewPosting
Commit	1b17b52 - feat: add employer job posting and new posting flow
Remote branch	origin/Employer/JobPosting-NewPosting
main	Not changed or pushed to

Pull request link: https://github.com/pragna2106-paduchuri/Waypoint/pull/new/Employer/JobPosting-NewPosting

If something stops working

- If Spring says port 8081 is already in use, stop the old BackendApplication before starting another one.

- If the browser shows an old UI, refresh with Ctrl+F5 so Vite reloads the latest bundle.

- If job requests return an employer-login error, log in again so localStorage contains a current accessToken.

- If MongoDB fails, confirm MONGODB_URI is configured in the IntelliJ run configuration.

In one sentence
React collects the job data, Spring Boot validates and owns the business rules, MongoDB stores the document, and the JWT proves which employer is allowed to manage it.